Desenvolvimento Web

Profª Sabrina B. Moreira

Fundamentos da web 3

Aula 3.1 Recapitulando as últimas aulas…

Internet: surgimento do hipertexto

- A internet é conhecida como a grande rede, e **interliga computadores de todo o mundo**.
- A partir das ideias de Tim Berners-Lee, surgiu o conceito de **hipertexto** que trouxe uma nova forma de organizar a informação.
- Permitia percorrer documentos por palavras destacadas no texto, chamado de **hyperlinks** ou **links**.

Internet: alguns marcos

1991	Surgimento da lista de discussão eletrônica WWW-talk, cujo foco é compartilhar experiências relacionadas com a linguagem HTML.
1992	Desenvolvimento do navegador Mosaic.
1993	Criação do navegador de texto Lynx.
1994	Estabelecimento da especificação HTML 2 (unificação das diferentes formas de HTML). Criação da empresa Netscape Communications Corporation. Surgimento do W3C (*World Wide Web Consortium*).
1995	Definição de novas marcações para o HTML. Criação do conceito de frames. Lançamento do navegador Internet Explorer 2.0.
1996	Lançamento do HTML versão 4.
1997	O HTML passa a incorporar tabelas, *applets*, subscritos, sobrescritos e texto combinado com imagens.
1998	Reformulação do HTML em XML.
1999	Lançamento do XHTML 1.0.
2004	Criação do grupo WHATWG (*Web Hypertext Application Technology Working Group*).
2007	Início do desenvolvimento do HTML 5, que incorpora TAGs para animações.

Fonte: MILETTO e BERTAGNOLLI, 2014

A arquitetura da web

- E como já vimos, a arquitetura web usa o modelo distribuído de **Cliente-Servidor** em conjunto com as **tecnologias web**, como o http e navegadores.

- Onde o cliente **solicita** informações ou **executa** ações num servidor utilizando um "endereço virtual" em um navegador.

- E o servidor **responde** com os **recursos** solicitados ou **confirmação das ações** realizadas pelo cliente.

Fundamentos da web 3

Aula 3.2 Conceitos importantes da web

Domínio

- O **endereço virtual** é informado a um software chamado navegador (browser), e a partir disso, ele acessa a página solicitada.

- Domínio é o **nome que identifica um site na internet**.

Fonte: registromarcas

Domínio

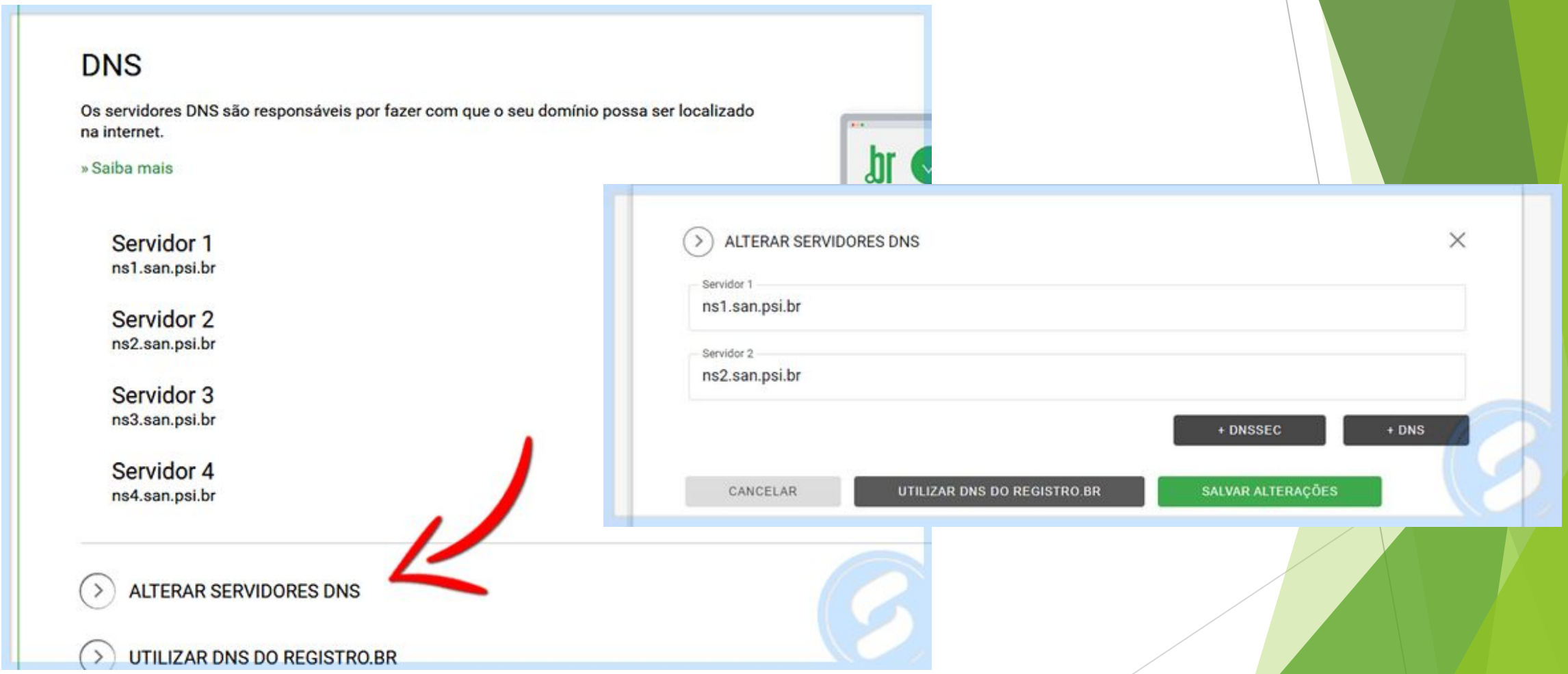
Exemplo de registradores no Brasil:

- Registro.br
- GoDaddy

Quando você registra um domínio:

- Ele é incluído no sistema mundial de domínios, coordenada pela ICANN (Internet Corporation for Assigned Names and Numbers).
- Você ganha acesso para configurar os servidores DNS.
- Você aponta o domínio para um IP.

Domínio: configuração no registro.br

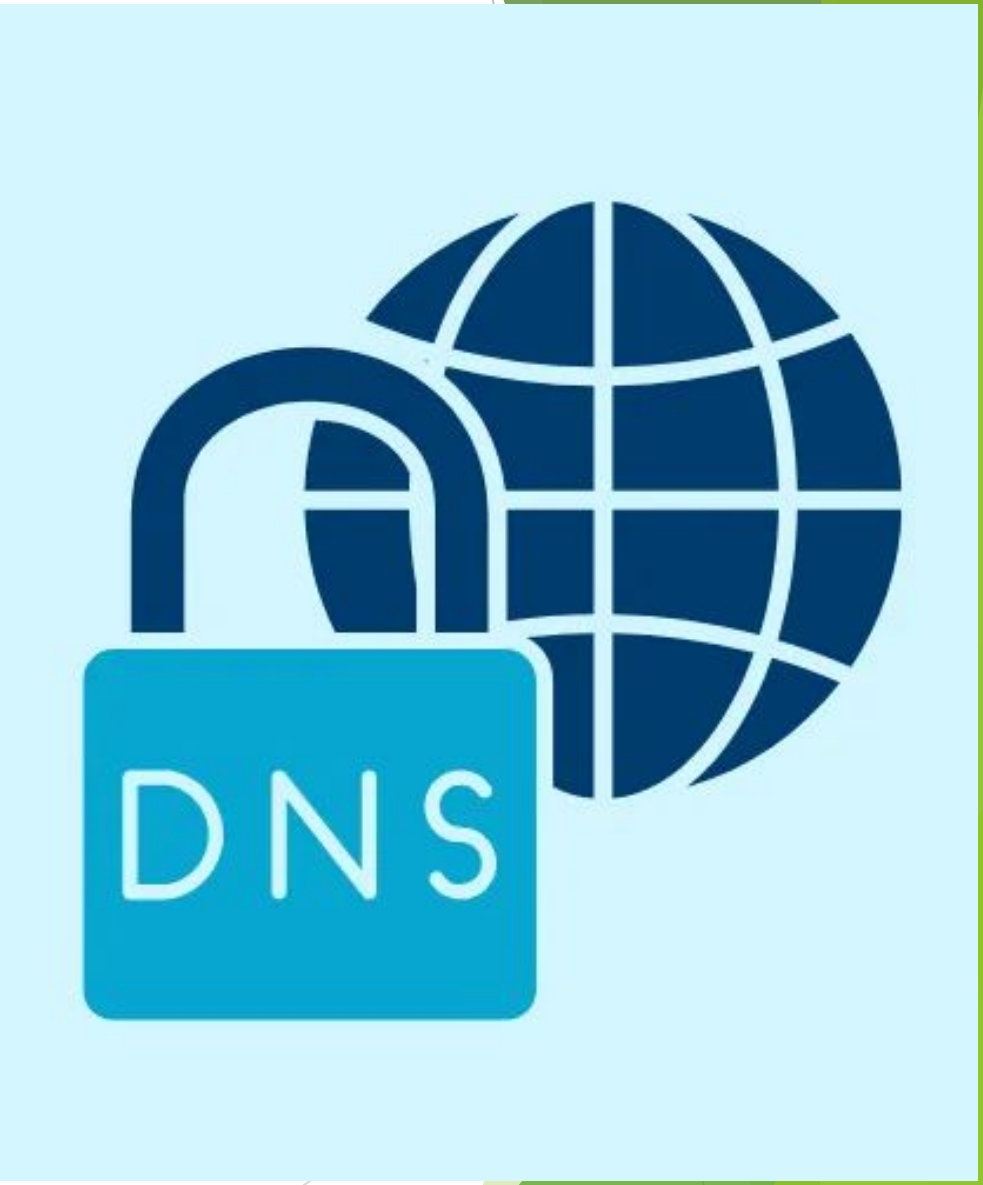


Fonte: saninternet

DNS (Domain Name System)

- Significa Domain Name System ou **Sistema de Nomes de Domínio.**

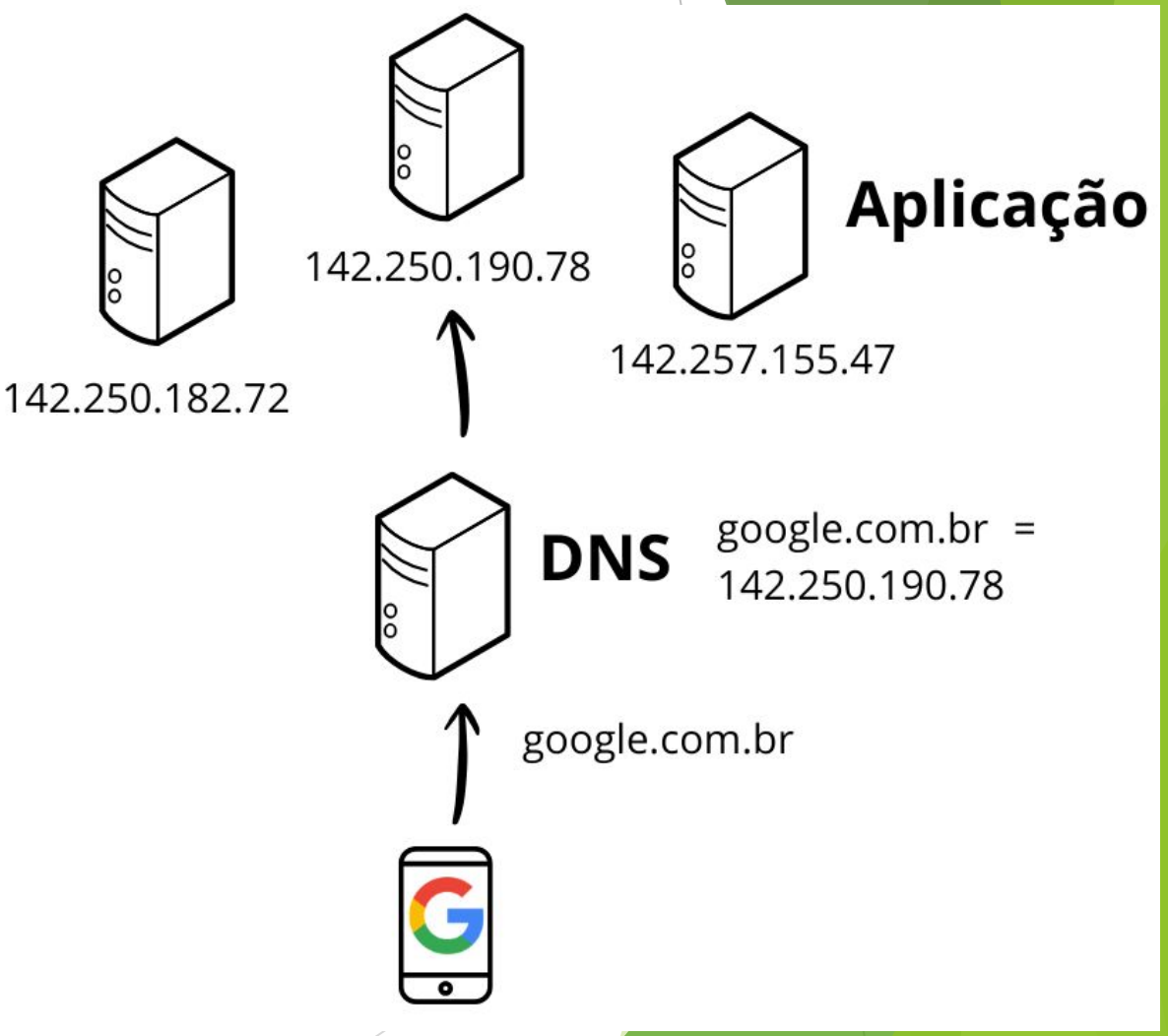
- Ele funciona como uma agenda telefônica da Internet, **buscando o IP do domínio** acessado pelos usuários.



Fonte: Lucciani, 2025

DNS (Domain Name System)

- Quando digitamos www.google.com.br, o cliente pergunta ao DNS qual é o endereço IP deste site.
- O DNS responde com um número (exemplo 142.250.190.78)
- O navegador usa esse IP para se conectar ao servidor onde o site está hospedado e receber os arquivos do site.



Fonte: autora, 2026

Mas por que precisamos do DNS?

- É muito **mais fácil memorizar** google.com.br do que um endereço IP como 142.250.190.78.

- Um mesmo servidor pode hospedar vários sites com o mesmo IP. O servidor web identifica qual site deve responder com **base no domínio** solicitado.

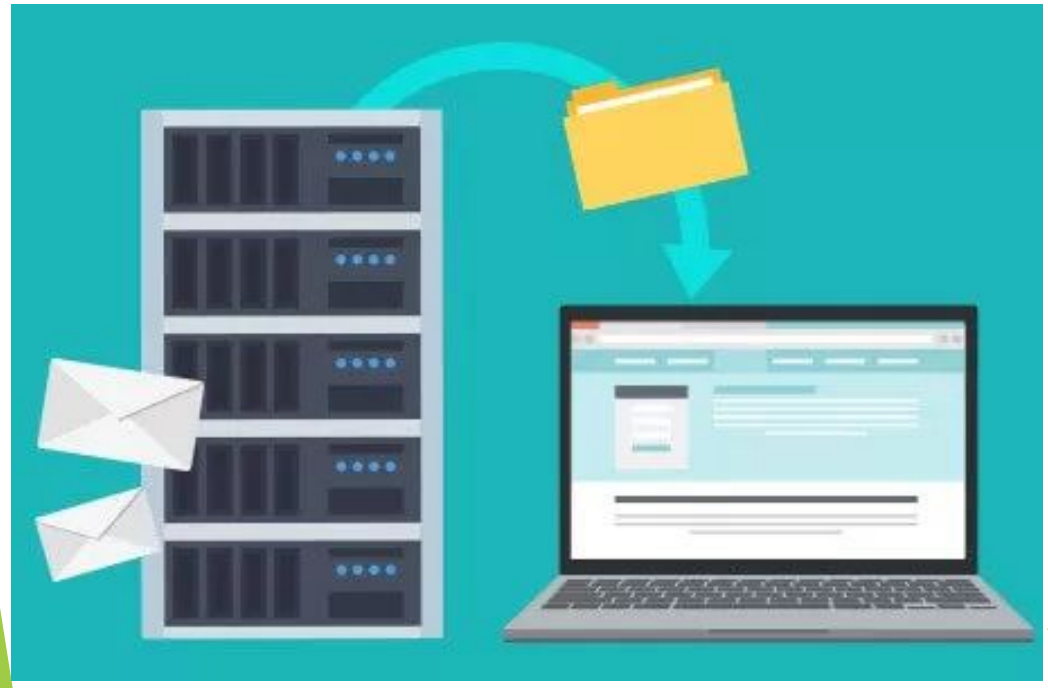
- Se o IP do servidor mudar, basta **atualizar o endereço no DNS**, e os usuários continuam acessando o site normalmente pelo domínio.

- Em sistemas grandes, uma aplicação pode estar **distribuída em vários servidores**. O DNS pode retornar diferentes endereços IP para ajudar a distribuir o tráfego entre eles.

Domínio x DNS

- O domínio é um **nome fácil de lembrar** associado a um site na internet.
- O sistema de DNS (Domain Name System) **converte esse nome em um endereço IP**, que é o identificador real do servidor na rede.
- O **domínio facilita o acesso** dos usuários aos sites, evitando que seja necessário memorizar endereços IP.
- O DNS, portanto, é o sistema responsável por **traduzir nomes de domínio em endereços IP**.

Hospedagem



Fonte: Delavy, 2026

- Quando criamos um site, ele precisa ficar armazenado em algum computador conectado à internet para que outras pessoas possam acessá-lo.

- Esse serviço é chamado de <u>hospedagem de sites</u>.

Hospedagem

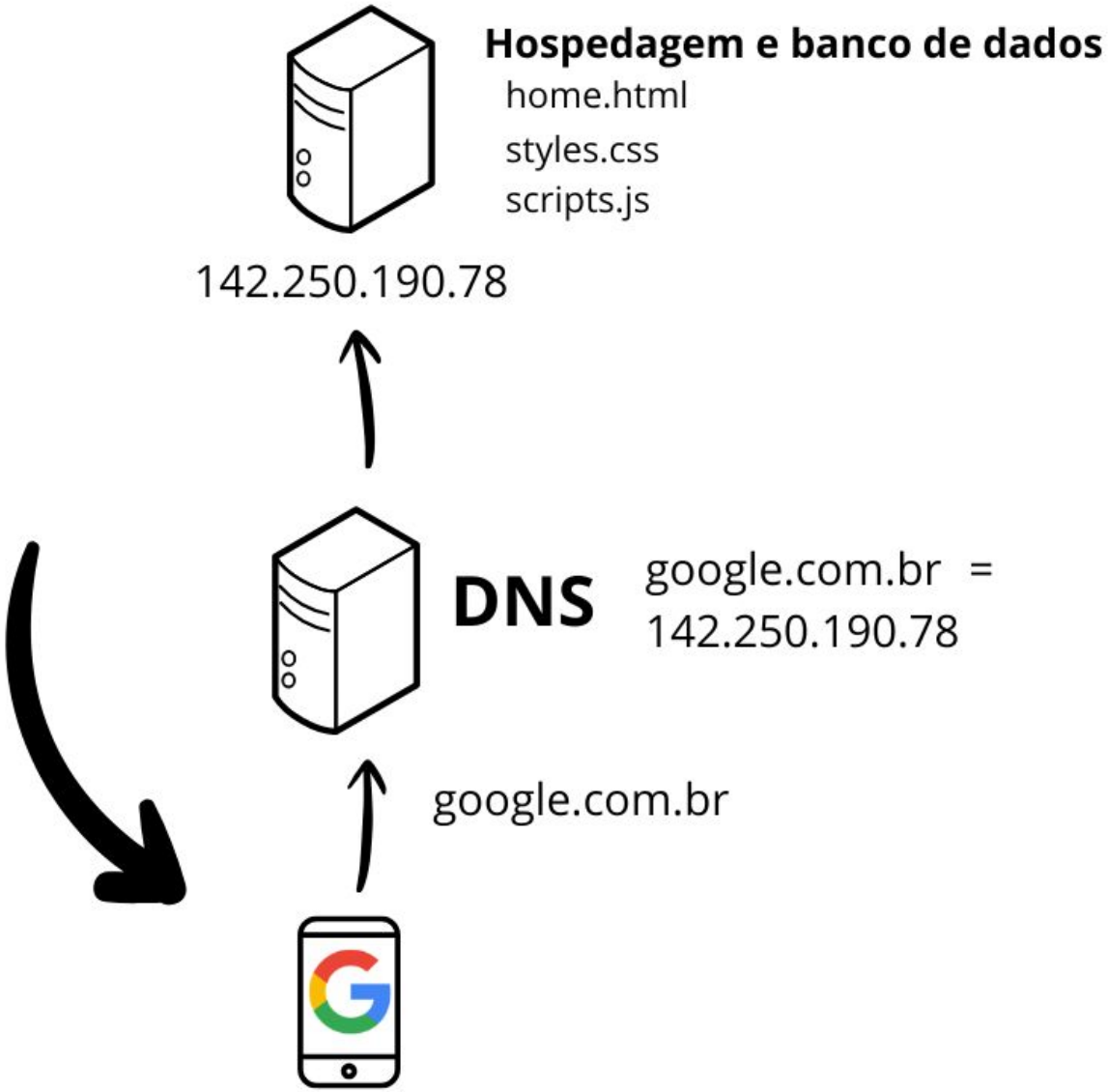
- A hospedagem disponibiliza **servidores conectados à internet que armazenam os arquivos** de um site.

- Permitindo sejam <u>acessados</u> usando o <u>navegador</u>.

Fonte: Delavy, 2026

Hospedagem

- O usuário digita o endereço do <u>site no navegador</u> para o acessar
- O DNS <u>converte o domínio</u> em um endereço IP
- O navegador <u>se conecta ao servidor</u> de hospedagem
- O <u>servidor envia os arquivos</u> do site ao navegador



Fonte: autora, 2026

Tipos de hospedagem

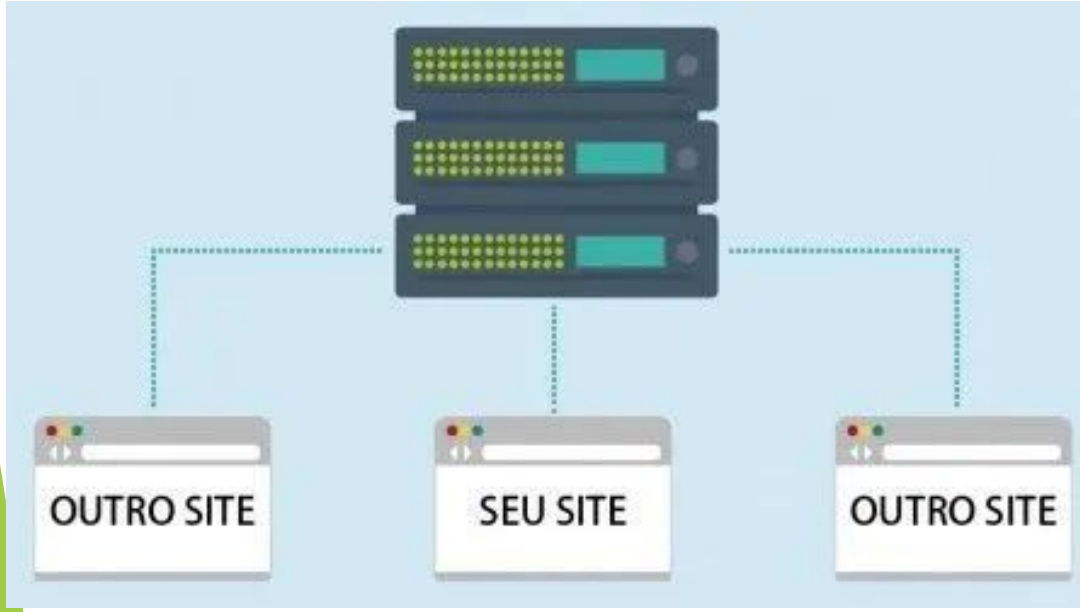
Arquitetura da aplicação x arquitetura de infra

- Uma **aplicação web** pode utilizar o modelo cliente-servidor e ainda assim estar hospedada em uma **infraestrutura centralizada** quando todos os componentes do lado do servidor estão em um único servidor.

- Se analisarmos apenas o lado do servidor e todos os seus componentes estiverem em uma **única máquina** de hospedagem, dizemos que a infraestrutura é centralizada.

- Quando falamos de hospedagem, estamos nos referindo ao ambiente onde a aplicação do **servidor** está **executando**.

Hospedagem compartilhada



Fonte: isbrasil, 2016

- **Vários sites utilizam o mesmo servidor físico,** compartilhando recursos como memória, processamento e armazenamento.

Cada cliente possui apenas uma área isolada para seus arquivos, mas o sistema operacional e os recursos do servidor são compartilhados entre todos.

Hospedagem compartilhada

- A arquitetura de infra é **centralizada**, porque todos os sites dependem de um único servidor. Se esse servidor tiver problemas, todos os sites hospedados nele podem ficar fora do ar.

- Utilizado para projetos simples, pois possui baixo custo e fácil administração. O provedor configura praticamente tudo, como servidor web, banco de dados e segurança básica.

- A principal limitação é que o desempenho pode variar, já que outros sites no mesmo servidor podem consumir muitos recursos, e o usuário tem pouco controle sobre a configuração do ambiente.

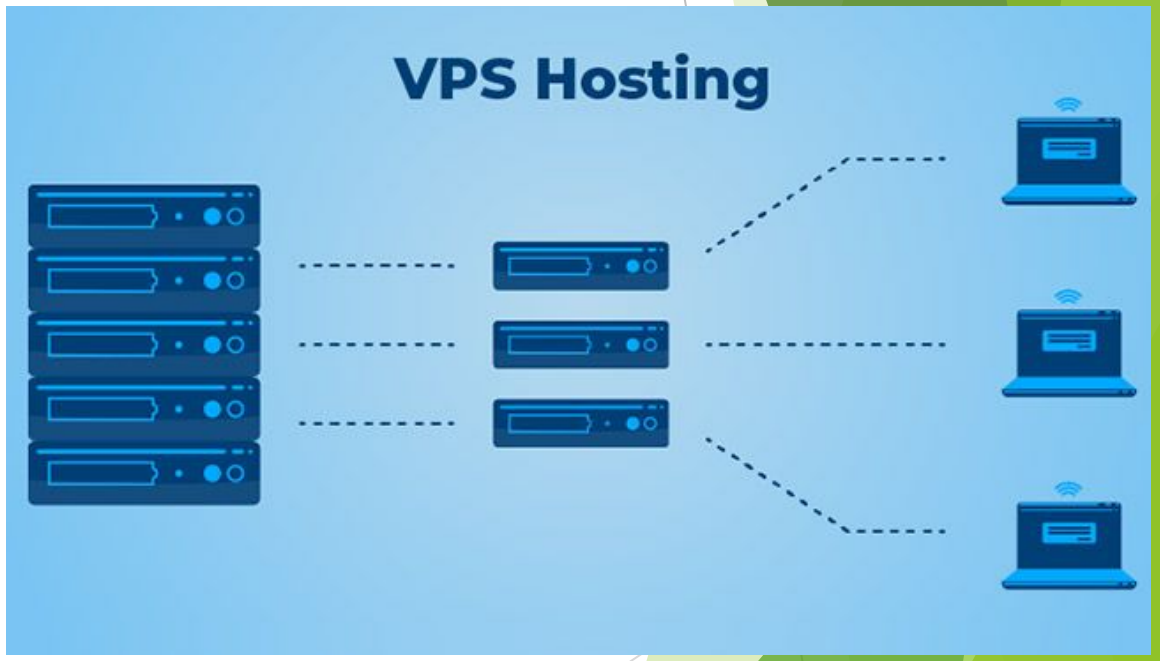
- A aplicação é um **monólito**, ou seja, todo o sistema roda em um único servidor, unindo aplicação e banco de dados no mesmo lugar.

VPS (Servidor Virtual Privado)

- Um **servidor físico** é dividido em várias **máquinas virtuais independentes** usando tecnologia de virtualização.

- Cada cliente recebe um servidor virtual com seu próprio sistema operacional, memória reservada e espaço em disco.

- Embora existam vários VPS no mesmo hardware físico, para o usuário ele funciona como um servidor próprio.



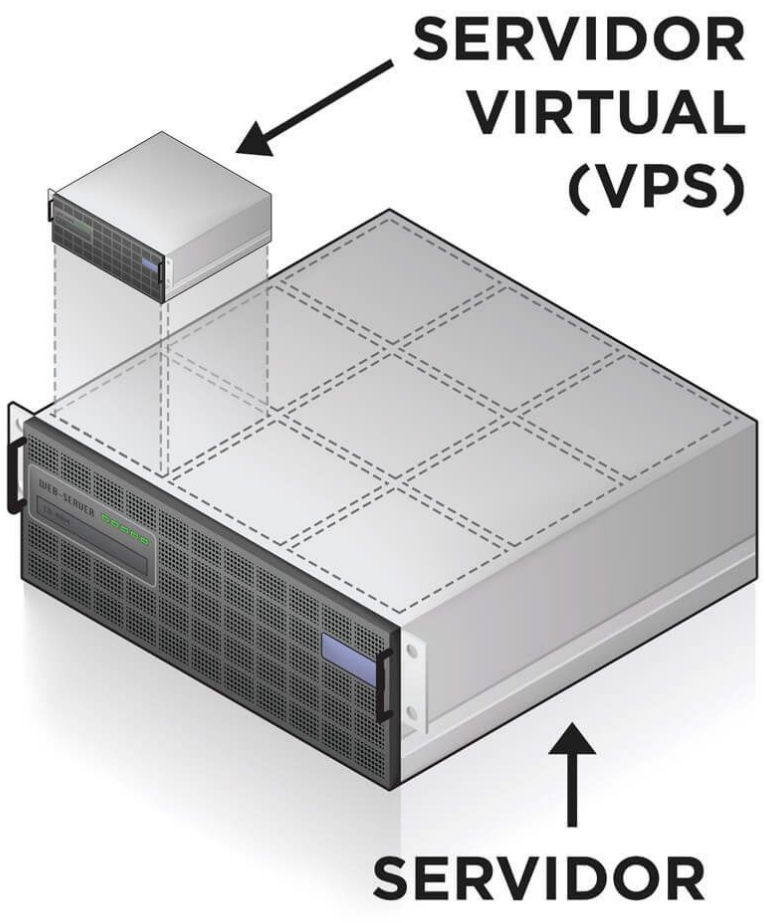
Fonte: hostbits, 2025

VPS (Servidor Virtual Privado)

- A arquitetura de infra é **centralizada**, pois cada VPS normalmente roda em uma única máquina virtual.

- Se essa máquina parar, o site hospedado nela fica indisponível.

VPS (Servidor Virtual Privado)



Fonte: hostcel

- O usuário tem **mais controle do ambiente**, podendo instalar softwares, configurar servidores web, banco de dados e outras ferramentas.

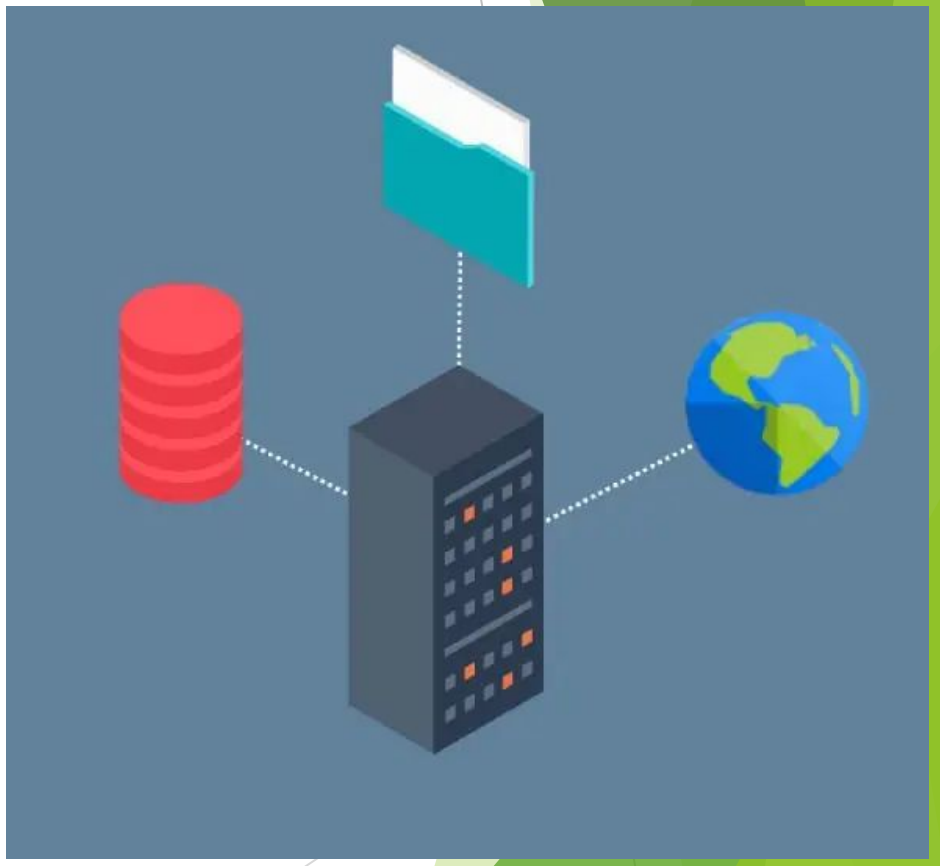
- O **desempenho é mais previsível** do que na hospedagem compartilhada. Mas exige mais conhecimento <u>técnico para administração</u> e possui <u>custo maior</u>.

Servidor dedicado

- Um cliente aluga **um servidor físico inteiro**, sem compartilhamento de hardware com outros usuários.

- Todos os recursos do servidor, processador, memória, armazenamento e rede são **utilizados exclusivamente por um único projeto** ou empresa.

- A arquitetura da infra é **centralizada**, onde toda aplicação está concentrada em um único servidor.



Fonte: task, 2016

Servidor dedicado

- A grande vantagem é ter o **máximo desempenho** e **controle total** do ambiente para configurar sistema operacional, instalar qualquer software e otimizar o servidor para suas necessidades.

- O custo é **elevado** e a administração exige conhecimento técnico em infraestrutura, segurança e manutenção de servidores.

- Muitas aplicações em servidores dedicados ainda utilizam uma arquitetura **monolítica**, com toda a aplicação rodando no mesmo servidor.

Hospedagem em nuvem

Fonte: Krinski, 2025

- A aplicação **não depende de um único servidor**, e sim, utiliza uma infraestrutura composta por <u>vários servidores conectados</u>, que podem **compartilhar carga** de processamento e armazenamento.

Hospedagem em nuvem

- A arquitetura de infra nesse caso é **distribuída**. Isso significa que a aplicação pode ser executada em múltiplas máquinas, permitindo maior disponibilidade e escalabilidade. Se um servidor falhar, outro pode assumir o processamento.

- A principal vantagem da nuvem é a **elasticidade**, a capacidade de aumentar ou reduzir recursos automaticamente conforme a demanda, lidando melhor com picos de acesso.

- Entretanto, a configuração pode ser mais **complexa**, e o **custo é variável** conforme o uso de processamento, armazenamento e tráfego de rede.

- É comumente usada para aplicações distribuídas utilizando micro serviços, mas também é possível usar para monólitos.

Hospedagem em nuvem

Principais grandes provedores de infraestrutura em nuvem:

- Amazon Web Services
- Google Cloud
- Microsoft Azure

Fonte: Ahmad, 2024

Publicação de projetos web

Processo de publicação de projeto web

Fonte: autora, 2026

Processo de publicação de projeto web

- **1. Compra do domínio**

Primeiro você registra um domínio em um serviço de registro de domínios, por exemplo: Registro.br

- **2. Contratação da hospedagem**

Depois você contrata um servidor para armazenar os arquivos do site, por exemplo: Hospedagem compartilhada na Hostgator.

Na hospedagem você terá:

- um servidor
- um endereço IP
- um painel de controle (cPanel ou similar)

Processo de publicação de projeto web

- **3. Configuração do DNS**

No painel do domínio no Registro.br você cria um registro DNS do tipo A:

Nome: meusite.com.br

Valor: 192.185.210.15

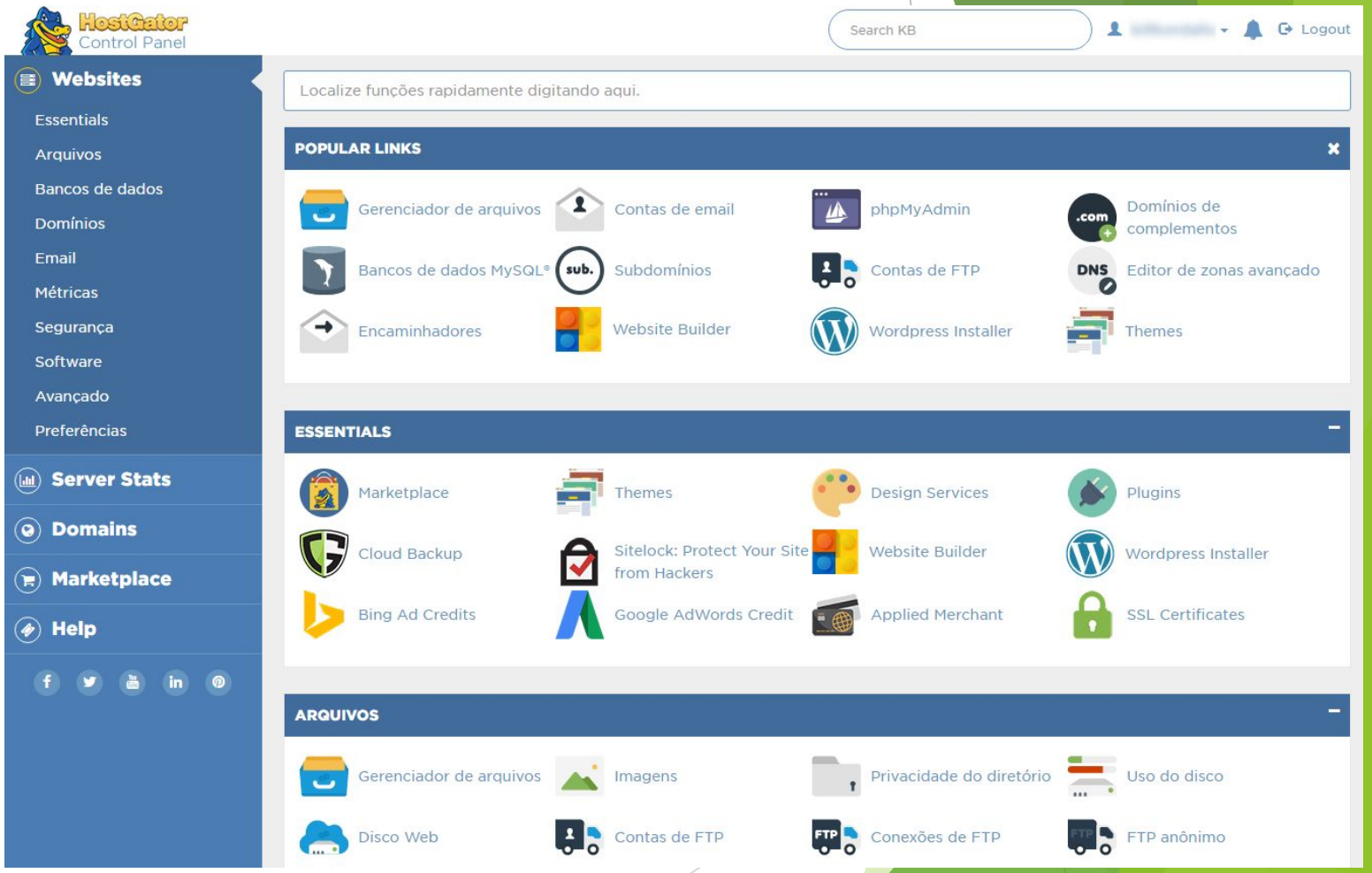
- Esse registro significa:

"Quando alguém acessar meusite.com.br, envie para o servidor 192.185.210.15."

Processo de publicação de projeto web

- **4. Upload do site**

Depois disso o desenvolvedor se conecta no servidor e <u>envia os arquivos</u> do projeto finalizado para o servidor devidamente configurado para os mesmos.



Fonte: hostigator

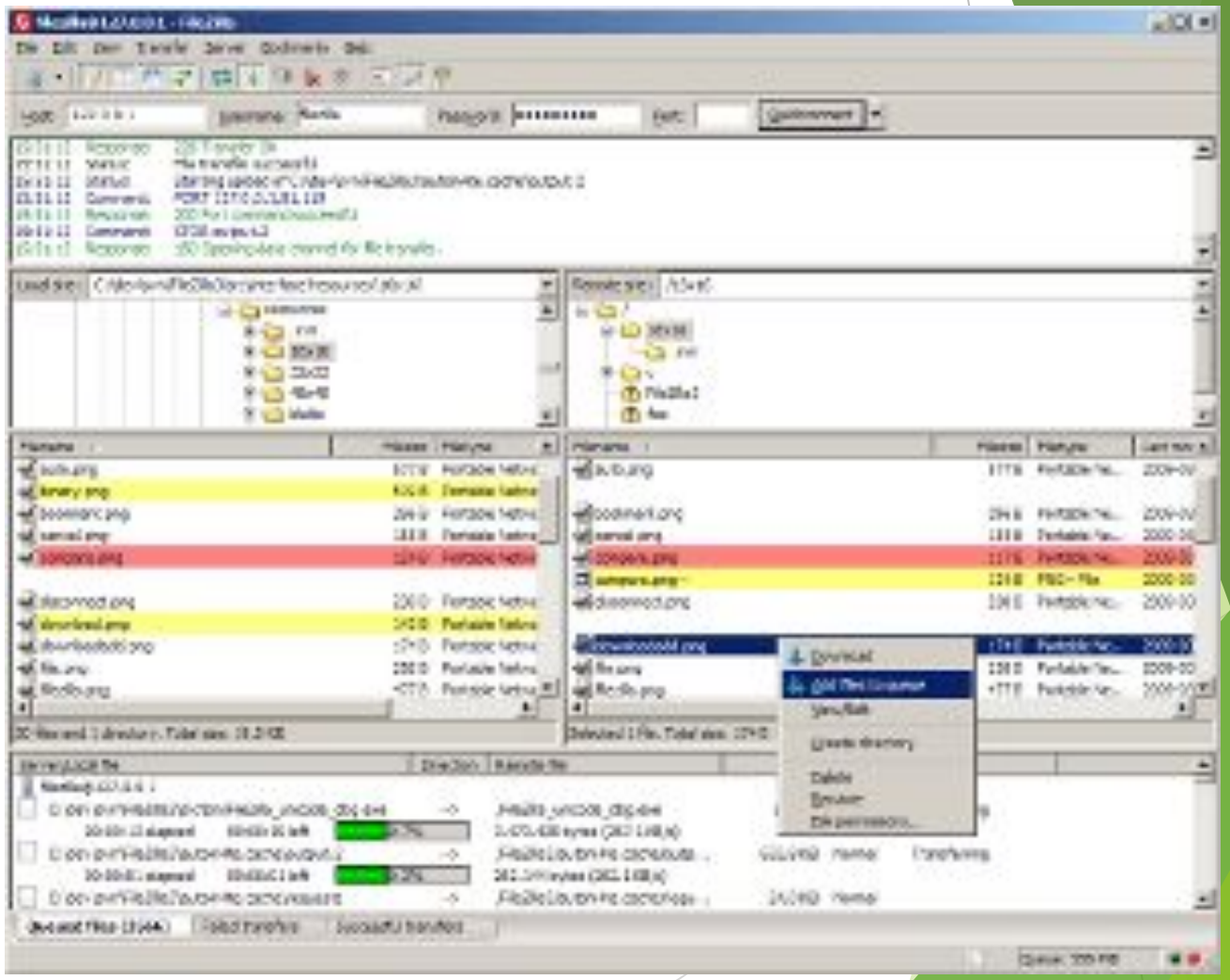
Processo de publicação de projeto web

Normalmente, as ferramentas para enviar arquivos ao servidor são:

- **FTP** (com softwares como Filezilla)
- **Git** (conectando diretamente no repositório da hospedagem)
- **File Manager do cPanel** (interface gráfica da hospedagem)

Processo de publicação de projeto web

- Transferência FTP via Filezilla



Fonte: filezilla

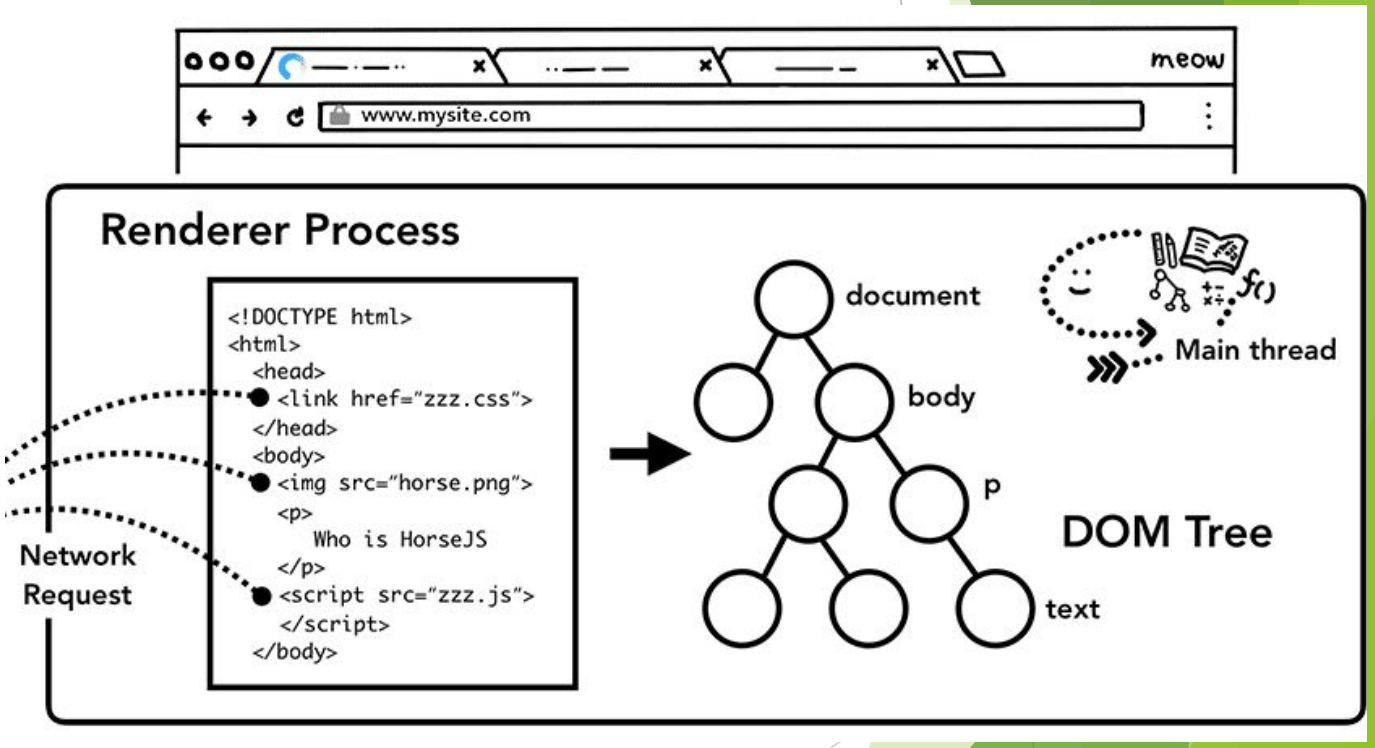
Navegadores (Browsers)

- Um navegador web é um software que permite **acessar** e **visualizar páginas da internet.**

Fonte: autora, 2026

Navegadores (Browsers)

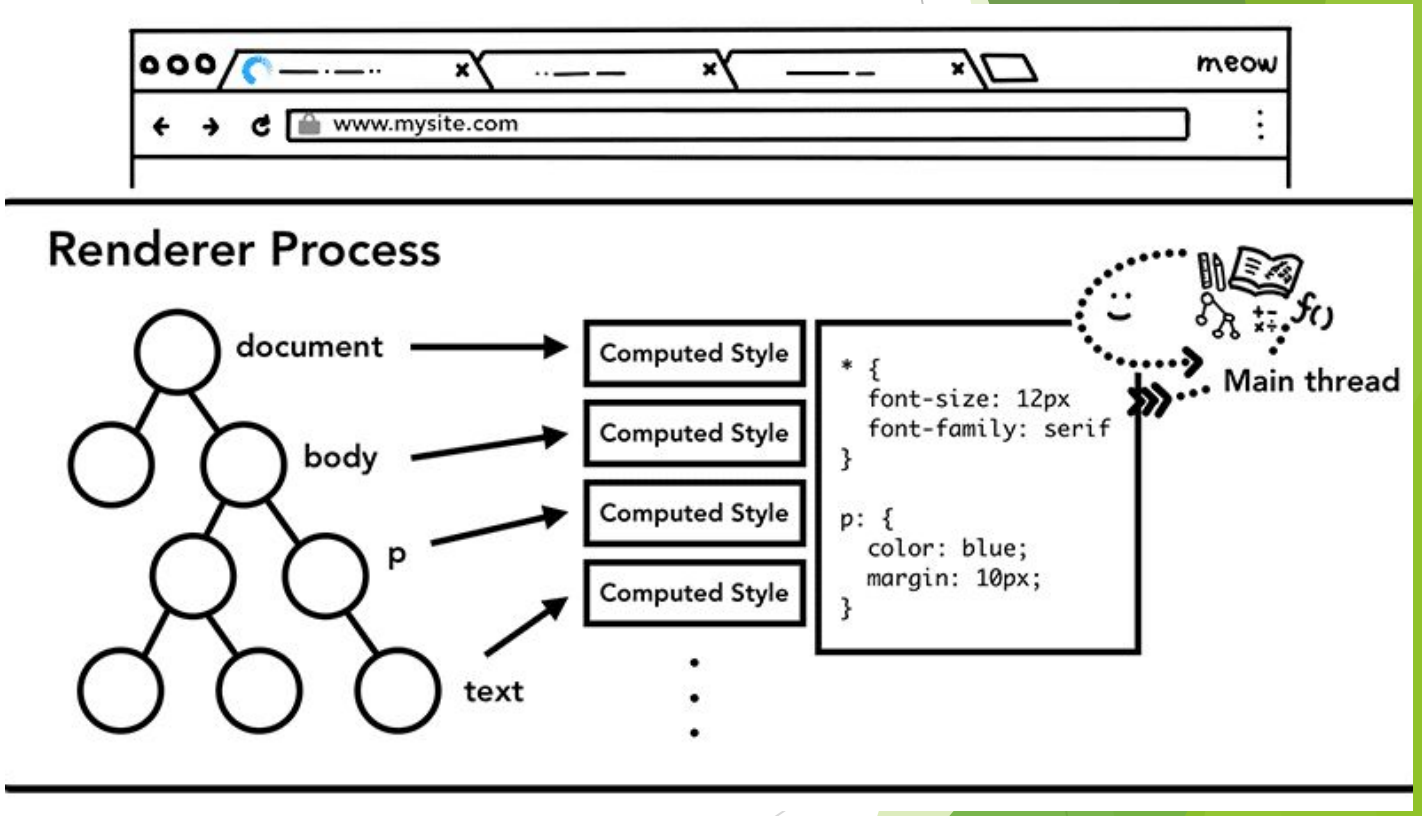
- Ele funciona como um **intermediário** entre o usuário e os servidores da web.
- **Interpretando os arquivos enviados pelo servidor** (HTML, CSS e JavaScript) e **exibindo o conteúdo de forma visual** na tela.



Fonte: Kosaka, 2018

Navegadores (Browsers)

- Desta forma, o navegador **transforma o código de um site em uma página** que podemos ler, clicar e interagir.



Fonte: Kosaka, 2018

URL (Uniform Resource Locator)

- A URL (Uniform Resource Locator) é o endereço completo de um recurso na internet.

Ela diz ao navegador:

- qual protocolo usar
- qual domínio acessar
- qual caminho dentro do servidor

URL (Uniform Resource Locator)

Fonte: MILETTO e BERTAGNOLLI, 2014

- Ele pode ser usado tanto na barra de endereços do navegador para acessar uma página ou usado como hiperlink, navegando para outra página.

Domínio x URL

- O domínio é como se fosse o "**nome fantasia**" de um site.

Em servidores dedicados, o domínio normalmente aponta para o IP exclusivo daquele servidor.

Em servidores compartilhados, vários domínios podem apontar para o mesmo IP. O servidor web identifica qual site deve ser exibido com base no domínio acessado pelo usuário.

- Já a URL é o **endereço completo** de um recurso no servidor.

Fundamentos da web 3

Aula 3.4 Protocolo HTTP

HTTP (HyperText Transfer Protocol)

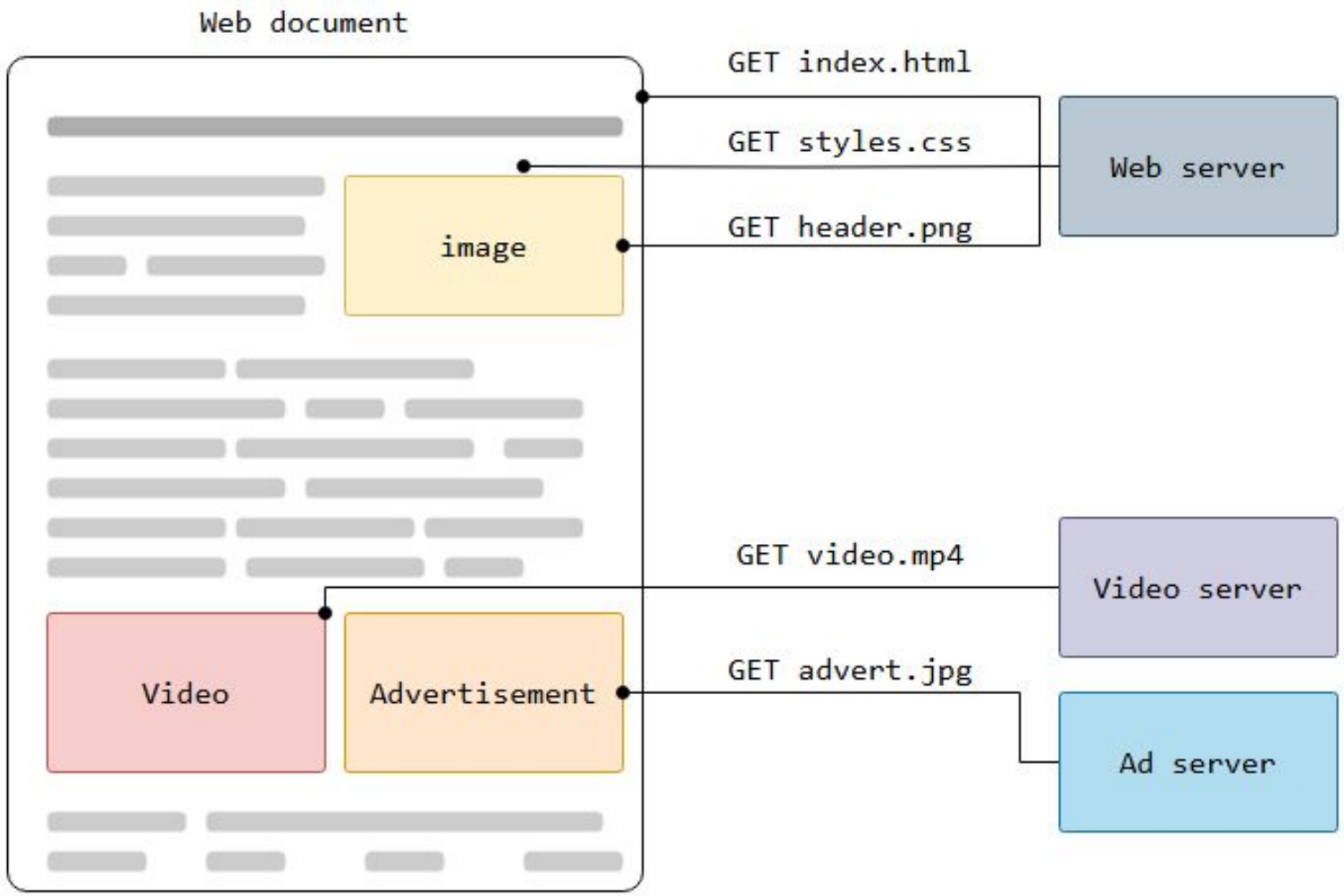
- O HTTP (HyperText Transfer Protocol) é o **protocolo** usado para **transferência de documentos hipermídia** (Hipertexto) na internet, e identifica o tipo de arquivo que será enviado. Surgiu em 1990.

- É a base de qualquer **troca de dados na Web** e um protocolo cliente-servidor, o que significa que as requisições são iniciadas pelo destinatário, geralmente um navegador da Web.

HTTP (HyperText Transfer Protocol)

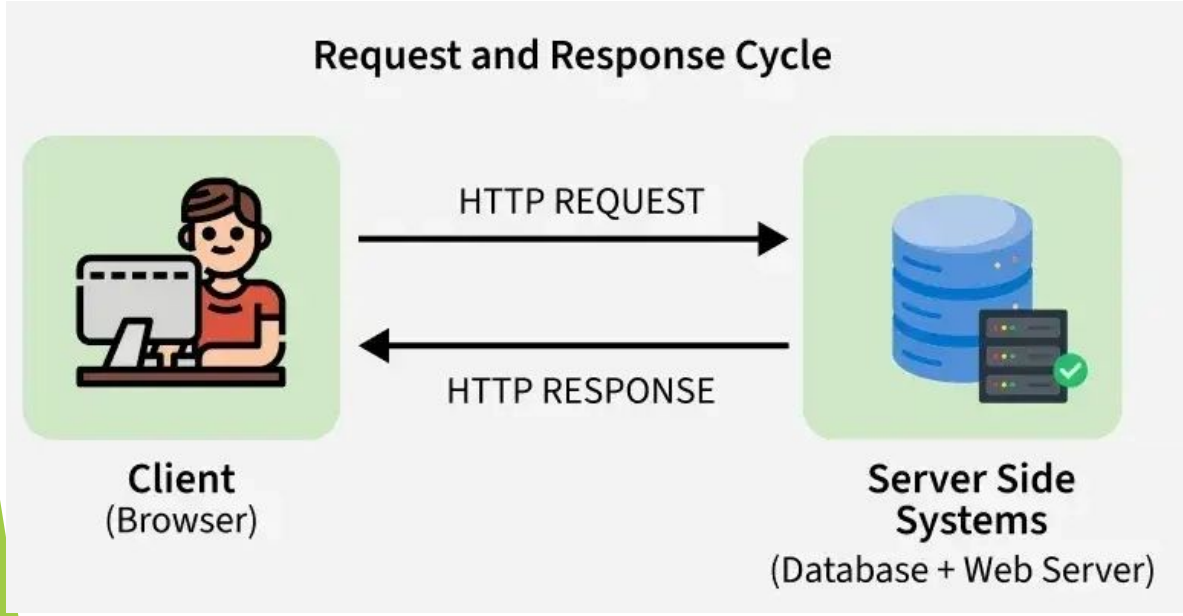
- Para mostrar uma página Web, o navegador envia uma **requisição** para buscar o documento HTML da página.
- Ele então realiza uma **análise sintática** desse arquivo, realizando requisições para obter os recursos da página (geralmente imagens e vídeos).
- Depois o navegador **interpreta** esses recursos para mostrar ao usuário a página completa.

HTTP (HyperText Transfer Protocol)



Fonte: developer mozilla org

HTTP: Request x Response



Fonte: Oduola, 2025

- As mensagens enviadas pelo <u>cliente</u>, são chamadas de **solicitações** (requests ou requisições) e as mensagens enviadas pelo <u>servidor</u> são chamadas de **respostas** (responses).

HTTP: agente

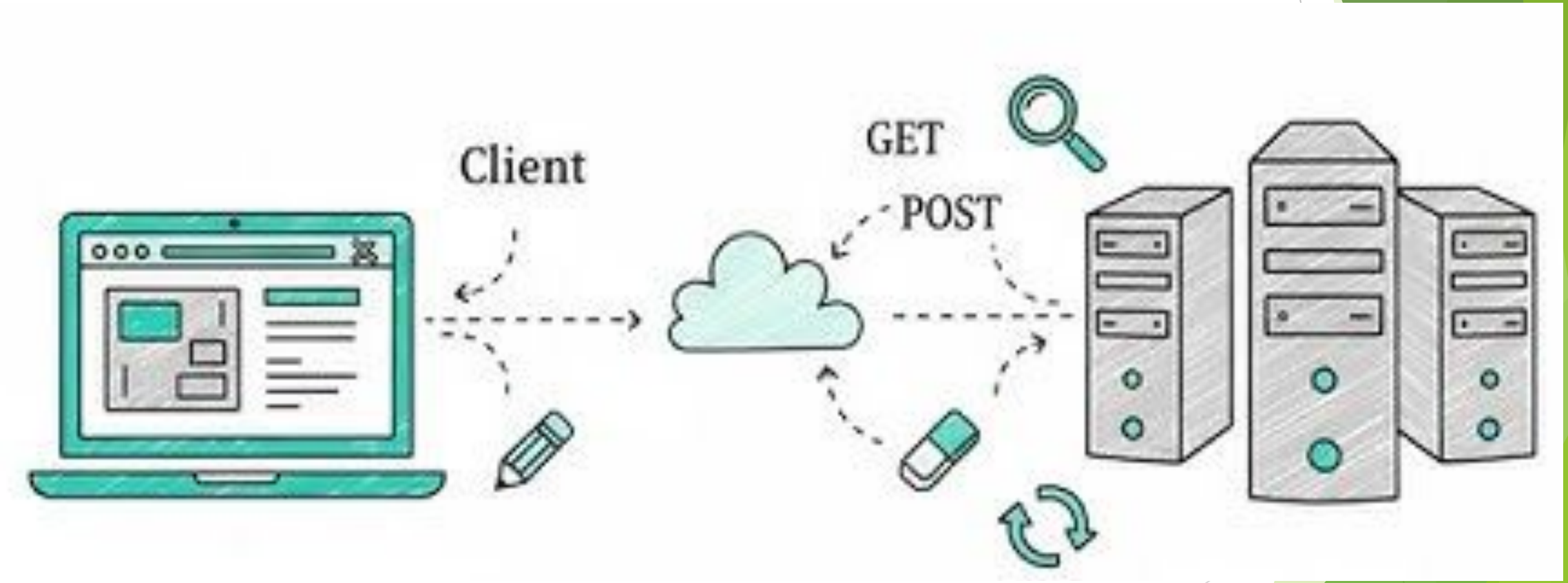
- Frequentemente, o **agente-usuário** é um navegador da Web, mas pode ser qualquer coisa.
- Por exemplo, como um robô que varre a Web para atualizar o índice de mecanismo de pesquisa.

E quem usa o HTTP?

- Navegadores (Chrome, Edge, Firefox)
- Aplicações frontend (JavaScript)
- Aplicativos mobile
- Serviços backend
- APIs

O HTTP é a **linguagem de comunicação entre sistemas** na web.

Principais métodos HTTP



Fonte: bits, 2026

Principais métodos HTTP

GET

- Usado para **solicitar** ou **consultar** um recurso no servidor.

Exemplo: carregar uma página ou buscar dados

GET /v1/products/iphone

- Buscar dados
- Não altera o servidor

Fonte: Nuzzi, 2024

Principais métodos HTTP

POST

- Usado para **enviar dados** ao servidor, geralmente para criar um novo recurso.

Exemplo: cadastrar um usuário

POST /v1/users

- Criar dados
- Envia informações no corpo da requisição

Fonte: Nuzzi, 2024

Principais métodos HTTP

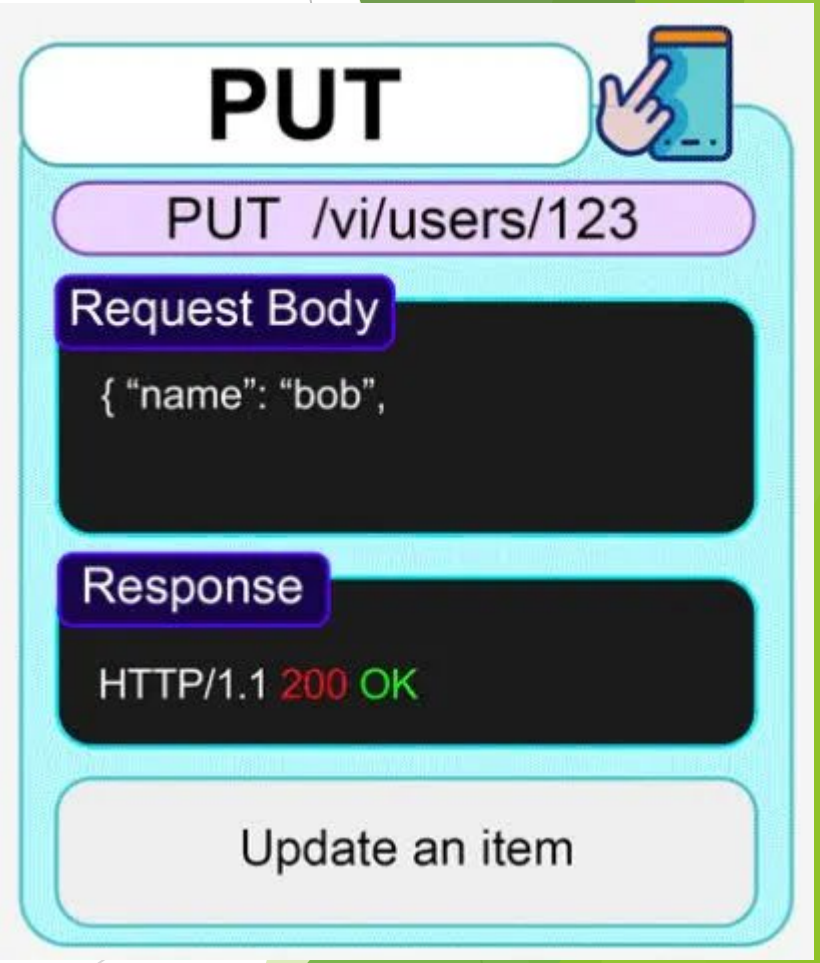
PUT

- Usado para **atualizar** <u>completamente</u> um recurso existente.

Exemplo: atualizar todas as informações de um usuário.

PUT /v1/users/123

- Atualiza todos os dados do usuário de ID 123.
- Envia informações no corpo da requisição



Fonte: Nuzzi, 2024

Principais métodos HTTP

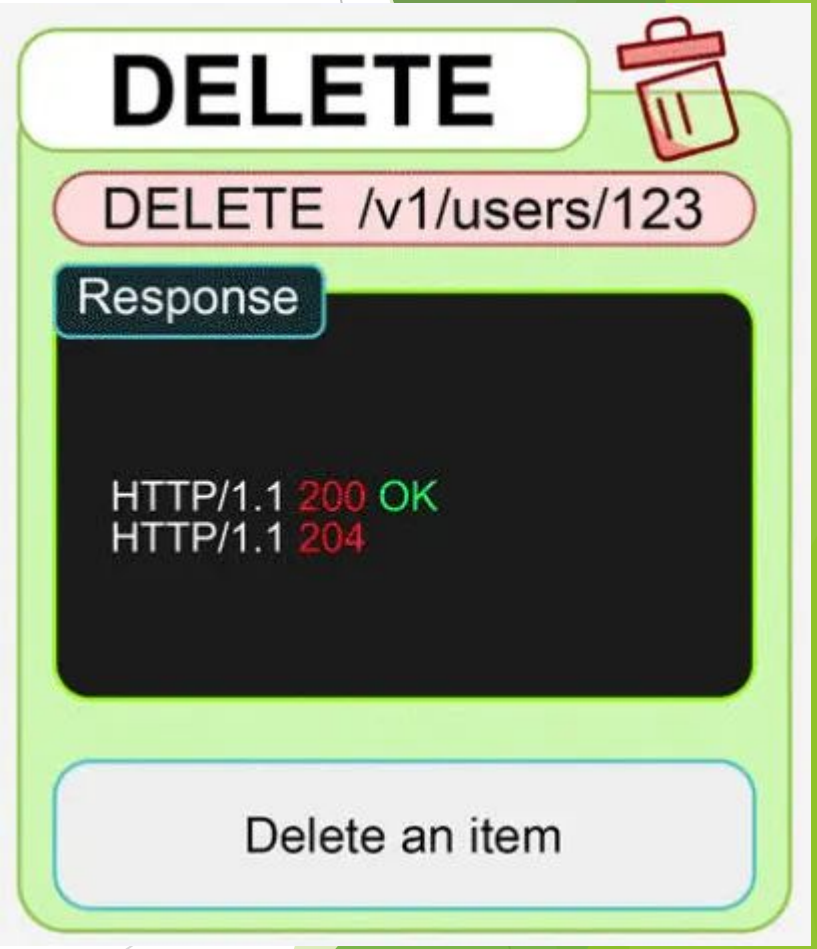
DELETE

- Usado para **remover** um recurso do servidor.

Exemplo: atualizar todas as informações de um usuário.

DELETE /v1/users/123

- Remove o usuário de ID 123.
- Geralmente a própria url define o que deverá ser deletado e não se envia o objeto a ser removido no corpo.



Fonte: Nuzzi, 2024

Outros métodos HTTP importantes

PATCH

- Usado para **atualizar** parcialmente um recurso.

Exemplo: alterar apenas o email de um usuário.

PATCH /v1/users/123

- Atualiza somente o dado do e-mail do usuário de ID 123.
- Envia informações no corpo da requisição

Fonte: Nuzzi, 2024

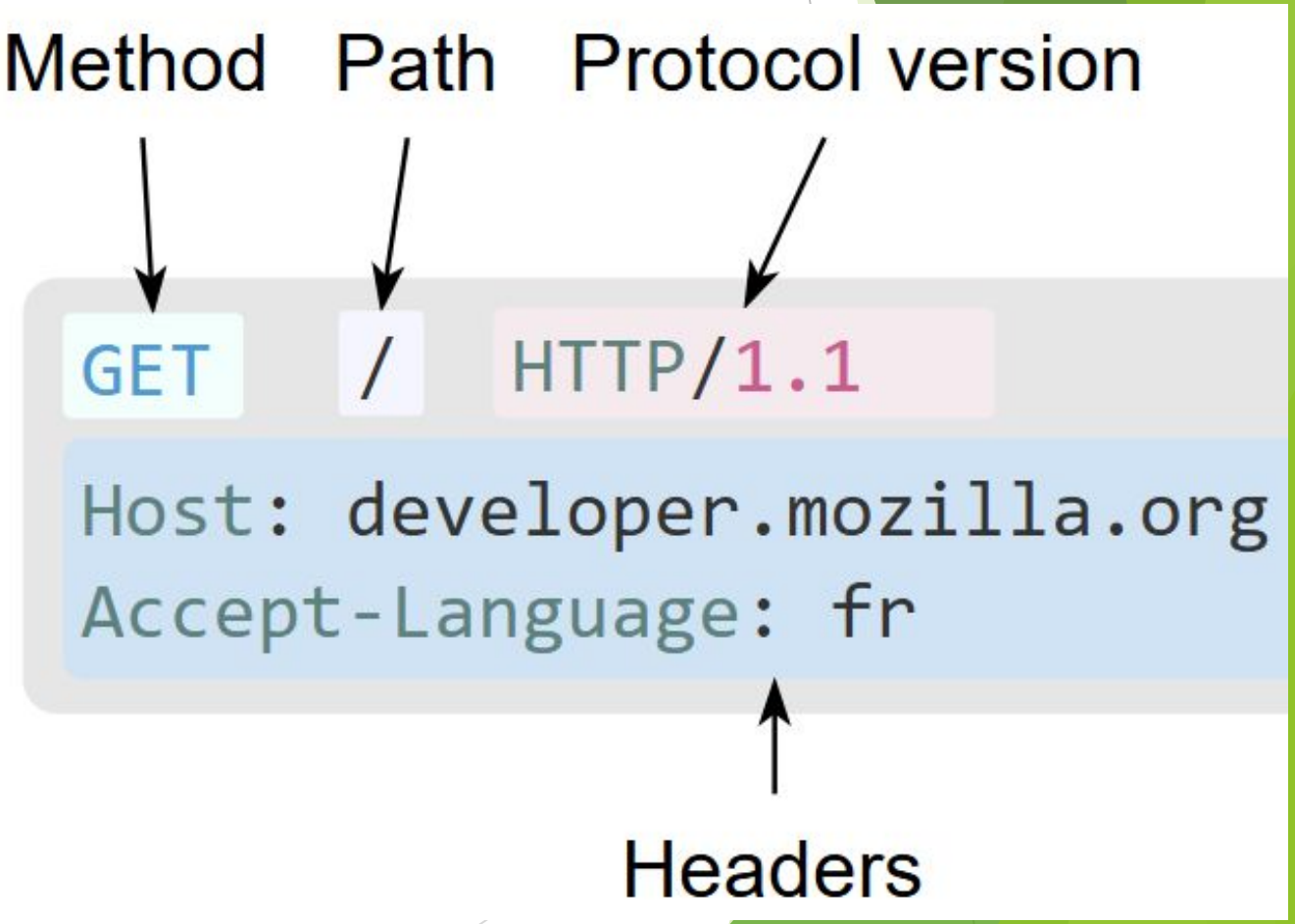
HTTP: estrutura da requisição

Uma requisição HTTP completa possui várias partes:

- Method (método): obrigatório
- URL / Endpoint: obrigatório
- Params (Query Parameters): opcional
- Headers (cabeçalhos): opcional
- Body: opcional

HTTP: estrutura da requisição

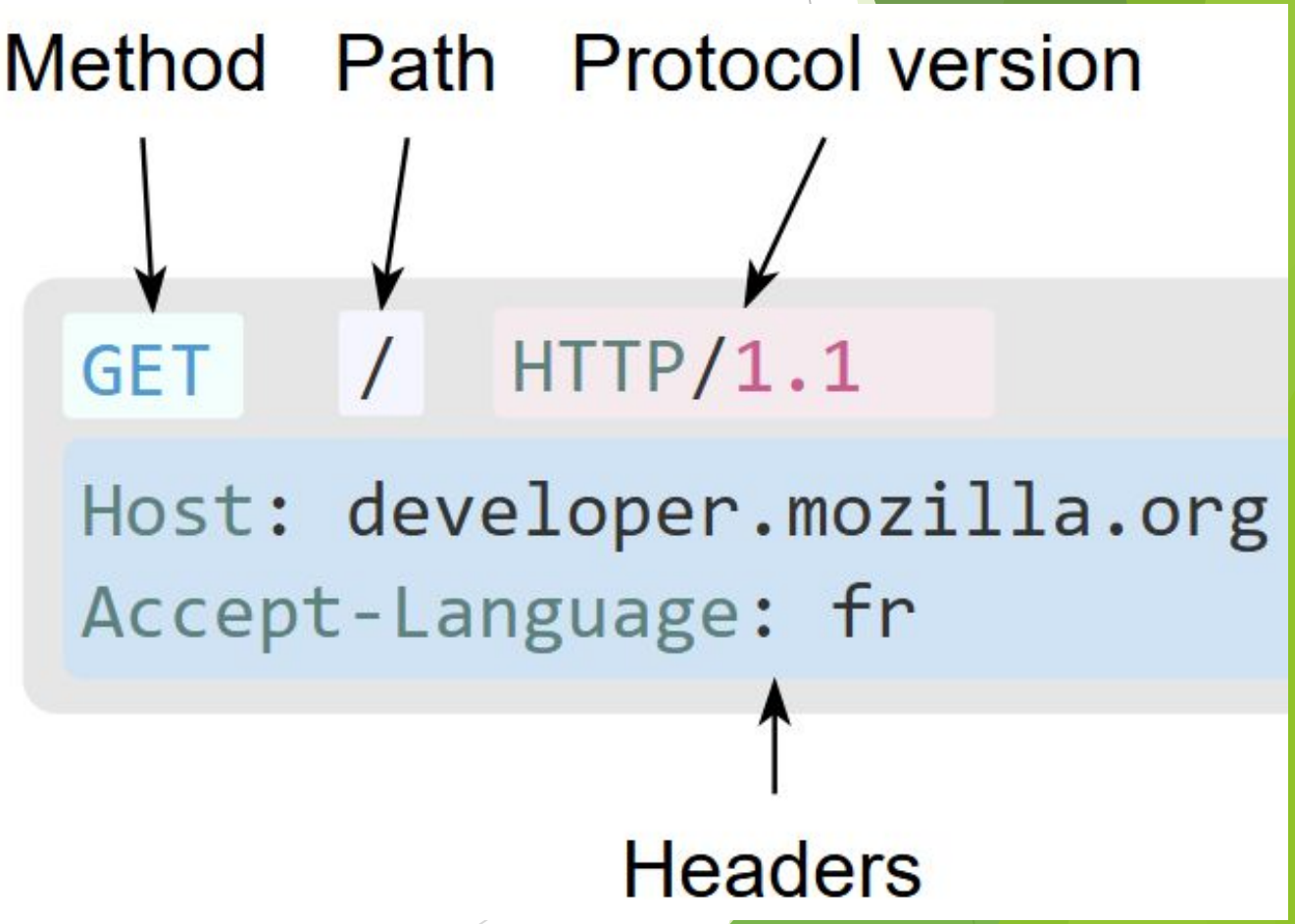
- **Método**: Geralmente é um <u>verbo</u> como GET, POST, DELETE, PUT ou <u>substantivo</u> como OPTIONS ou HEAD.



Fonte: developer mozilla org

HTTP: estrutura da requisição

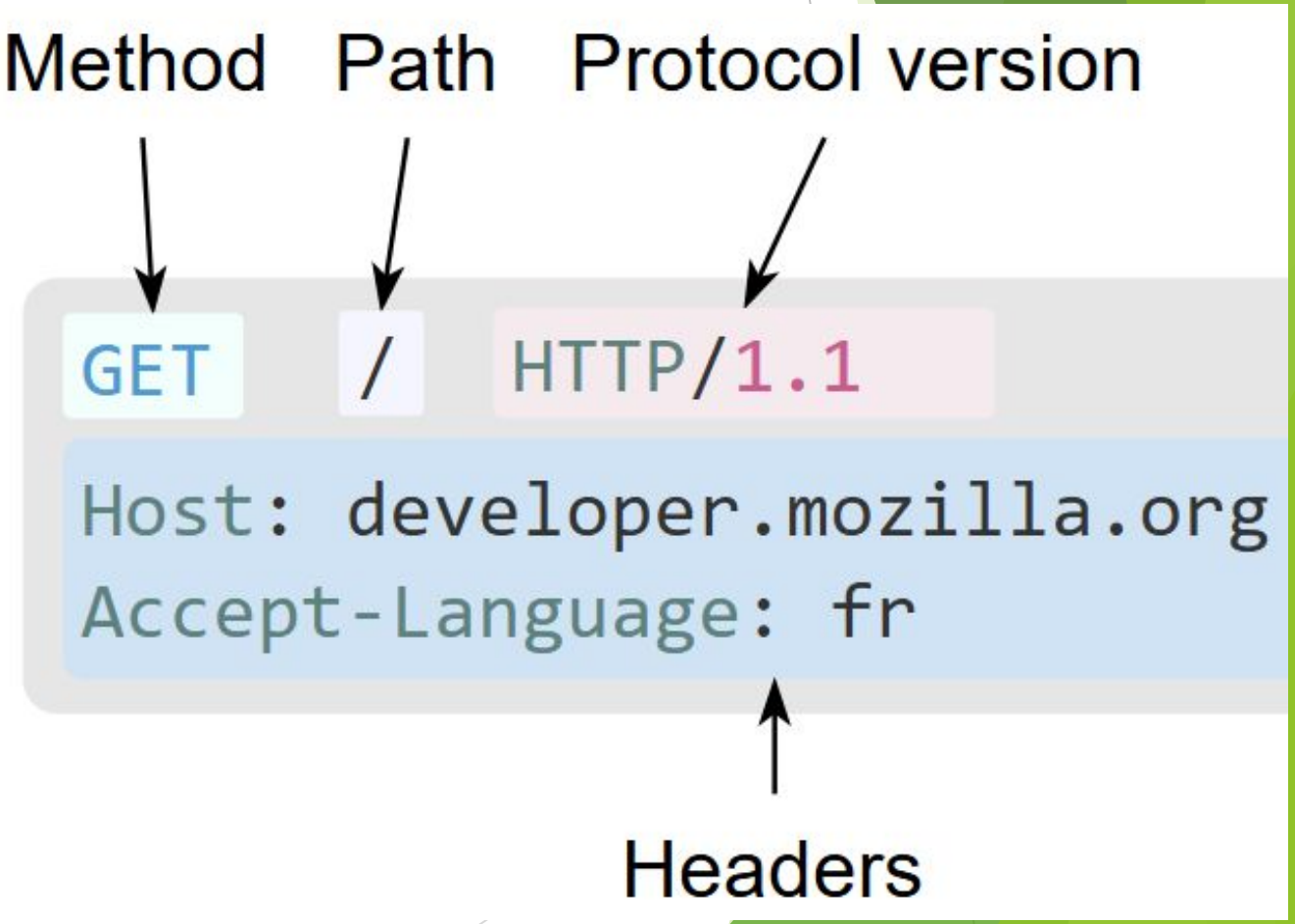
- **Caminho**: A URL do recurso sem os elementos do contexto (protocolo, domínio e porta).



Fonte: developer mozilla org

HTTP: estrutura da requisição

- **Versão:** A versão do protocolo HTTP utilizada.



Fonte: developer mozilla org

HTTP: estrutura da requisição

Podemos ter informações opcionais como:

- **Cabeçalhos** (headers) da requisição, que contém informações adicionais, como tipo de conteúdo, autenticação e outras configurações.

- **Conteúdo** (body) da requisição, que é o corpo contendo dados enviados ao servidor, geralmente utilizado em métodos como POST, PUT ou PATCH.

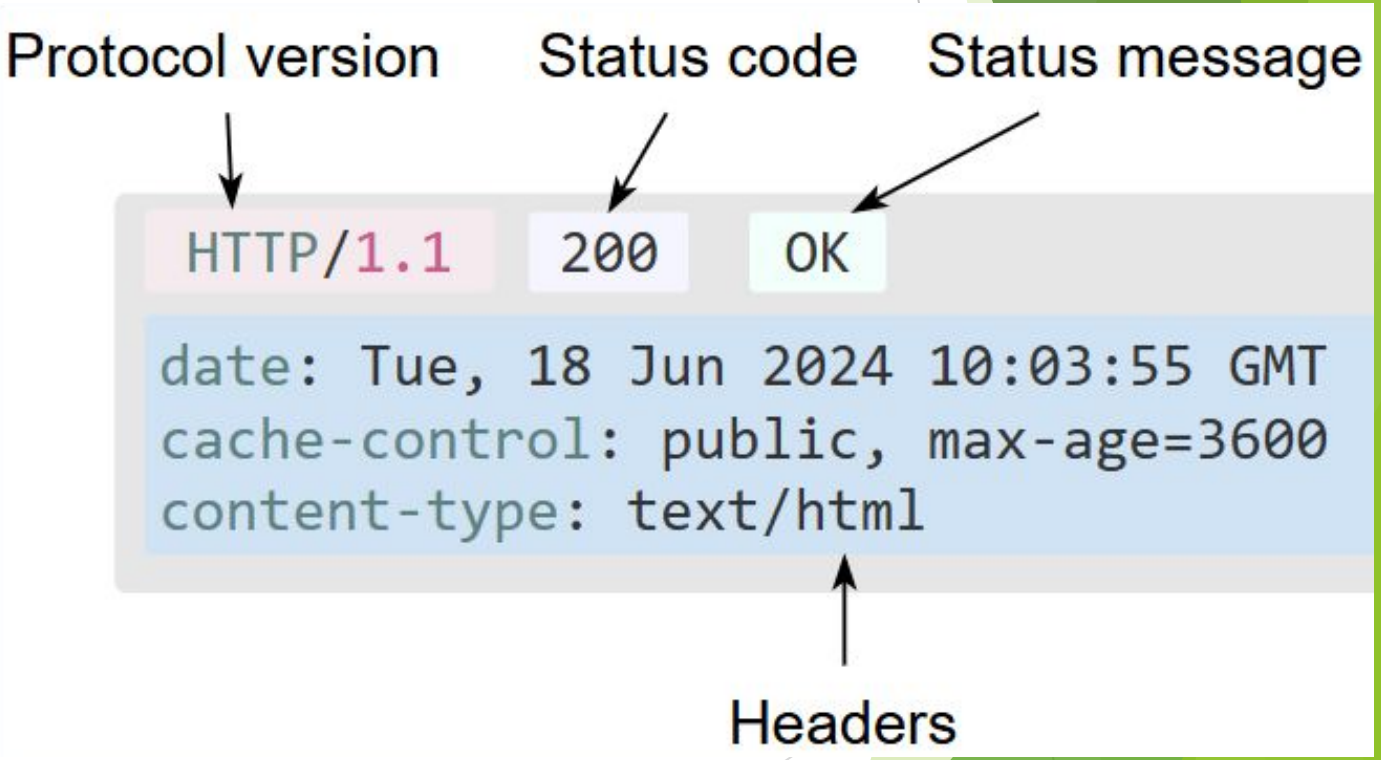
HTTP: estrutura da resposta

A resposta da API também possui partes, principalmente:

- **Código de status** (Status Code): obrigatório
- **Cabeçalhos** da resposta (Headers): opcional
- **Conteúdo** da resposta (Body): opcional

HTTP: estrutura da resposta

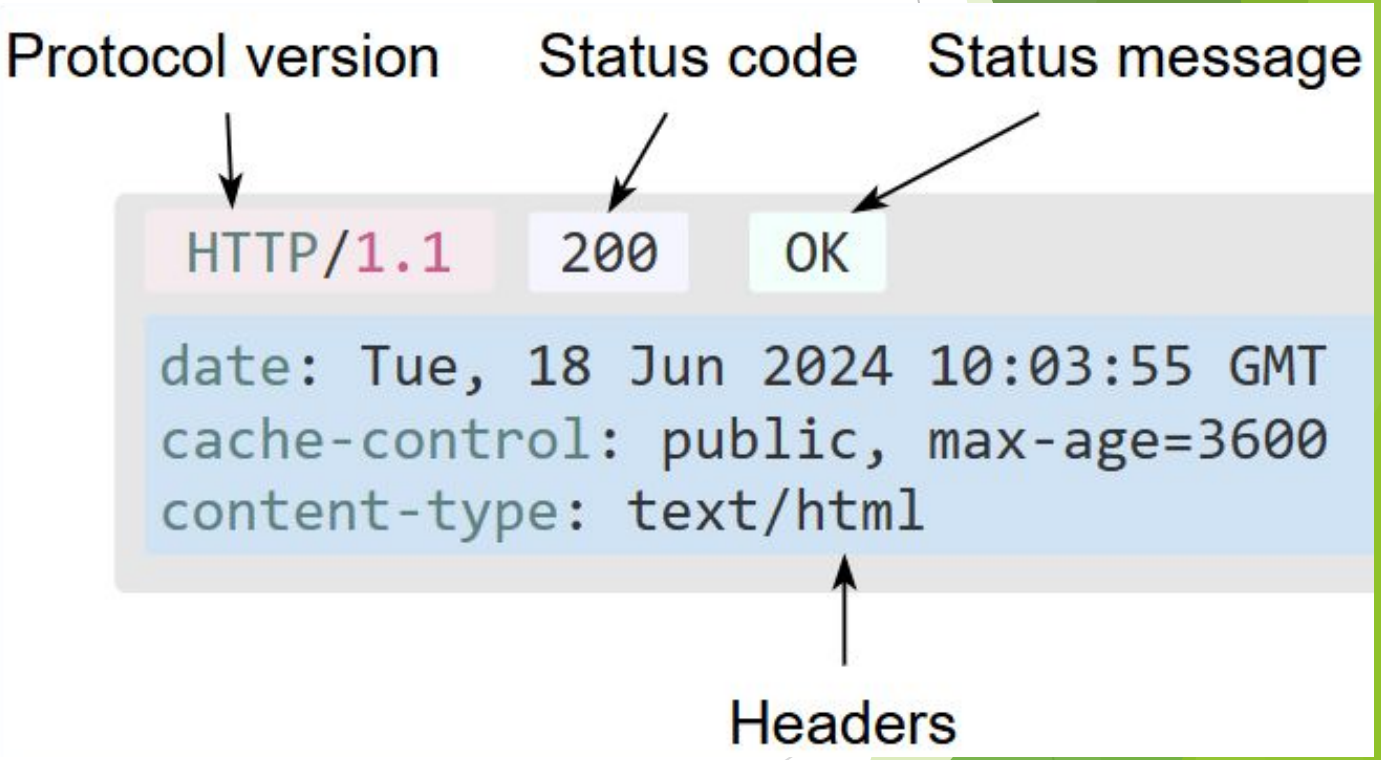
- **Versão**: A versão do protocolo HTTP utilizada.
- **Código de status**: códigos que indicam se a requisição foi bem sucedida ou não.
- **Mensagem de status**: pequena descrição do código de status.



Fonte: developer mozilla org

HTTP: estrutura da resposta

- Também podemos ter os **cabeçalhos** e **corpo** da resposta como conteúdo <u>opcional</u>, similar ao que vimos na requisição.



Fonte: developer mozilla org

Código de status das requisições HTML

Os códigos de status indicam se uma requisição HTTP foi executada com **êxito** ou **não**.

Sendo agrupadas em 5 classes:

- Respostas Informativas (100 – 199)
- Respostas bem-sucedidas (200 – 299)
- Mensagens de redirecionamento (300 – 399)
- Respostas de erro do cliente (400 – 499)
- Respostas de erro do servidor (500 – 599)

Principais código de status de sucesso

200 OK: A solicitação foi bem-sucedida.

- **GET**: O recurso foi obtido e transmitido no corpo da mensagem.
- **HEAD**: O recurso foi obtido e o cabeçalho foi enviado.
- **PUT** ou **POST**: O resultado da ação é transmitido no corpo da mensagem.
- **TRACE**: o diagnóstico foi realizado com sucesso e no corpo da mensagem é retornado a requisição feita.

Fonte: Nuzzi, 2024

Principais código de status de sucesso

201 Created

- A requisição foi bem sucedida e um novo recurso foi criado como resultado.

- Esta é normalmente a resposta enviada após as solicitações **POST** ou algumas solicitações **PUT**.

Fonte: Nuzzi, 2024

Principais código de status de erro

400 Bad Request

- O servidor não pode ou não irá processar a solicitação devido um erro do cliente (por exemplo, sintaxe de solicitação malformada).

401 Unauthorized

- Embora o padrão HTTP especifique "unauthorized", semanticamente, essa resposta significa "unauthenticated".
- Ou seja, o cliente deve se autenticar para obter a resposta solicitada.

Principais código de status de erro

403 Forbidden

- O cliente não tem direitos de acesso ao conteúdo, não é autorizado, portanto o servidor está se recusando a fornecer o recurso solicitado.

- Ao contrário do 401 Unauthorized, aqui a identidade do cliente é conhecida pelo servidor, mas ele não ter permissão para acessar.

Principais código de status de erro

404 Not Found: O servidor não pode encontrar o recurso solicitado.

- No navegador, isso significa que o **URL não é reconhecido**.
- Os servidores também podem enviar esta resposta em vez de 403 Forbidden **para ocultar a existência de um recurso** de um cliente não autorizado.

Principais código de status de erro

405 Method Not Allowed

- O método de solicitação é conhecido pelo servidor, mas não é suportado pelo recurso de destino.

Exemplo: o endpoint é um get, mas tento passar como delete.

O servidor entende o método DELETE, mas ele **não é permitido para esse endpoint**, então a resposta pode ser 405.

Principais código de status de erro

429 Too Many Requests

- Indica que o cliente enviou requisições demais ao servidor em um período muito curto de tempo.
- Para evitar sobrecarga ou abuso do sistema, é implementado o mecanismo de limitação de frequência (rate limiting).
- Define quantas requisições um usuário ou aplicação pode fazer em determinado intervalo de tempo.

Principais código de status de erro

500 Internal Server Error

- O erro HTTP 500 (Internal Server Error) indica que ocorreu um problema inesperado no servidor, e ele não conseguiu processar a requisição feita pelo cliente.
- O servidor tentou executar a operação solicitada, mas encontrou um erro interno que não conseguiu tratar adequadamente.

Principais código de status de erro

501 Not Implemented

- Indica que o servidor não reconhece ou não implementa aquele método HTTP.
- Ou seja, o próprio servidor não possui suporte para esse tipo de requisição, independentemente de qual recurso esteja sendo acessado.

Exemplo: Se um cliente envia uma requisição usando o método PATCH, mas o servidor foi configurado para suportar apenas GET e POST, ele responde com 501 Not Implemented.

Métodos que todo servidor deve suportar

De acordo com a especificação do protocolo HTTP, todo servidor deve ser capaz de lidar pelo menos com os métodos:

- GET
- HEAD

Por isso, um servidor <u>não deveria</u> retornar 501 para esses dois métodos, pois eles são considerados básicos para o funcionamento da web.

Fonte: Nuzzi, 2024

Principais código de status de erro

503 Service Unavailable

- Indica que o servidor está temporariamente indisponível para processar a requisição.
- Isso acontece quando o servidor está funcionando, mas não consegue atender à solicitação naquele momento.
- Normalmente esse problema é temporário, e o serviço volta a funcionar depois de algum tempo.

Principais código de status de erro

Fonte: freepik

O erro **503** pode ocorrer por vários motivos, como:

- Manutenção do servidor
- Sobrecarga de acessos (muitos usuários ao mesmo tempo)
- Recursos insuficientes (memória, CPU ou conexões)
- Reinicialização ou atualização do sistema

Dúvidas?

Atividade 3 - Questionário

- Responder questionário referente aos assuntos da aula de hoje.
- **Data de entrega**: até o final da aula de HOJE
- **Forma de entrega**: manuscrito físico.
- **Execução**: duplas ou trios (colocar nome de todos)
- **Fonte de consulta**: slides e colegas

Referência da aula

- LUCCIANI, Geraldo. Quanto tempo levam as alterações de DNS no Registro.br? Veja. TASK, 2025. Disponível em: https://www.task.com.br/blog/quanto-tempo-demora-alteracoes-registro-br/. Acesso em: 5 mar. 2026.

- Mariko Kosaka. Inside look at modern web browser (part 3). Google, 2018. Disponível em: https://developer.chrome.com/blog/inside-browser-part3?hl=pt-br. Acesso em: 5 mar. 2026.

- MOZILLA. Códigos de status de respostas HTTP. MDN Web Docs. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Status. Acesso em: 5 mar. 2026.

Referência da aula

- MILETTO, Evandro M.; BERTAGNOLLI, Silvia C. Desenvolvimento de software II: introdução ao desenvolvimento web com HTML, CSS, javascript e PHP. (Tekne). Porto Alegre: Bookman, 2014. E-book. p.72. ISBN 9788582601969. Disponível em: https://integrada.minhabiblioteca.com.br/reader/books/9788582601969/. Acesso em: 08 mar. 2026.

- NUZZI, Jones Roberto. 9 métodos de requisição HTTP. Medium, 22 mar. 2024. Disponível em: https://medium.com/@jonesroberto/9-m%C3%A9todos-de-requisi%C3%A7%C3%A3o-http-20159bd6ee6c. Acesso em: 12 mar. 2026.

univille

univille.br